Đinh Hải Triều

AI INTERN

Phone: 0878883316 Mail: trieudh14@gmail.com Address: Bắc Từ Liêm(old), Hà Nội

CAREER OBJECTIVE

- A third-year IT student passionate about Generative AI and Computer Vision. Seeking an **AI Intern** position to leverage my experience in building RAG systems and deep learning models. Aiming to contribute to scalable AI solutions while refining skills in model optimization and production-grade AI deployment.
- **Github**: https://github.com/trieu123x
- **Portfolio**: https://portfolio-puce-one-rissyoxuv7.vercel.app

EDUCATION

Posts and Telecommunications Institute of Technology (PTIT)
Information Technology | GPA: 2.96 / 4.0 | 2023 – Present

SKILLS

- **AI & Machine Learning:** LLMs (Gemini, GPT), RAG (Retrieval-Augmented Generation), Prompt Engineering, Vector Search (Cosine Similarity), Object Detection (YOLOv8), Image Classification (ResNet50), HuggingFace Transformers, Sentence Transformers.
- **Model Evaluation:** mAP, F1-Score, Precision/Recall, Confusion Matrix, Train/Val Accuracy.
- **Frameworks & Libraries:** PyTorch, FastAPI, Flask, OpenCV, Ultralytics, NumPy, Pandas.
- **Data & Vector DB:** pgvector (PostgreSQL), Supabase, Gemini text-embedding-004.
- **Backend & Frontend:** Python (Advanced), Node.js, ReactJS, RESTful API, SSE (Streaming).
- **Tools:** Git/GitHub, Docker, Postman, Kaggle GPU, Linux.

PROJECT

K-Hospital – MediCare: Full-Stack Hospital Appointment Platform (01/2026 – 05/2026)(Team 3)

- **RAG Workflow (Owner):** Designed an end-to-end RAG pipeline including query rewriting, intent classification, hybrid vector search (pgvector + keyword), and context-aware prompt construction, followed by real-time LLM response streaming with Gemini fallback handling.
- **Context Injection Strategy:** Implemented intelligent logic to determine when to include doctor recommendation context versus plain message, balancing response quality and API cost efficiency.
- **Data Chunking Pipeline:** Built preprocessing pipeline to chunk disease documents (symptoms, descriptions, treatment plans) into optimal segments before embedding and storing vectors in PostgreSQL with pgvector for efficient semantic retrieval.
- **Streaming & Reliability:** Handled full connection lifecycle with proactive stream cancellation on client disconnect to prevent unnecessary Gemini API token consumption; implemented asyncio.CancelledError edge case handling.
- **Backend Integration:** Collaborated on Node.js/Express backend logic calling the AI service; defined shared API contracts between backend and FastAPI AI module.
- **Tech Stack:** Python 3.12, FastAPI, Google Gemini API, pgvector (PostgreSQL), Server-Sent Events, asyncio, Node.js, Next.js, Prisma, Supabase.
- **Github**: https://github.com/trieu123x/k-hospital
- **Deploy**: https://k-hospital.vercel.app

Real-Time Garbage Classification System – 12 Classes (03/2026 – Present)(Solo)

- **Two-Stage Pipeline:** Integrated custom-trained YOLOv8 for object detection and bounding box localization, feeding cropped ROIs into a fine-tuned ResNet50 classifier with letterbox padding for aspect-ratio-preserving preprocessing (+15% context margin).
- **Model Training:** Fine-tuned YOLOv8 and ResNet50 (ImageNet pretrained) on TACO-based datasets using PyTorch with CosineAnnealingLR and balanced sampling; achieved mAP@50: 0.83 (detection) and accuracy: 0.84 (classification).
- **Deployment:** Served via Flask REST API supporting file-upload and URL-based inference; returned base64-encoded annotated images with per-class confidence scores; implemented lazy model loading via @app.before_request to prevent server startup blocking.
- **Technical Challenges:** Resolved nn.DataParallel module prefix mismatch for single-GPU/CPU inference compatibility; tuned independent confidence thresholds for detection (0.15–0.25) and classification (0.10–0.60) stages; added User-Agent spoofing to bypass CDN 403 blocks on URL-based inference.
- **Real-Time Video:** Built OpenCV-based realtime inference pipeline with per-frame detection and classification annotation; supports video file input.
- **Tech Stack:** Python, PyTorch, YOLOv8 (Ultralytics), ResNet50, OpenCV, Flask, PIL, CUDA/CPU fallback, Kaggle Notebooks.
- **Github**: https://github.com/trieu123x/Garbage_detect_and_classify